Implicit and Explicit Default Comparison Operators

Document number: P0432R1
Date: 2016-09-18
Reply-to: David Stone
Audience: Evolution Working Group

Changes since P0432R0

- Fixed some typos
- Operators are only generated if the operators that they forward to are not deleted
- User-defined comparison operators only inhibit implicit operator== if both arguments are of that type
- Added a possible simplified syntax for defaulting operators, inspired by Bravely Default (P0481R0)
- Added section on ODR use and hiding
- Added examples
- Deleted open question about the definition of operator<
- Added open question about how hiding should work.

Background

Previous attempts at trying to add defaulted comparison operators to the language have generally been divided into two groups, implicit vs. explicit.

The explicit proposal (N3950) requires users to opt-in using something like this:

```
struct S {
    bool operator==(S const &) = default;
    bool operator!=(S const &) = default;
    bool operator<(S const &) = default;
    bool operator>(S const &) = default;
    bool operator<=(S const &) = default;
    bool operator>=(S const &) = default;
};
```

For small classes, this is almost as verbose as just defining the operators manually. In practice, users would create something like `#define GENERATE_COMPARISON_OPERATORS(type)` to somewhat automate this boilerplate.

The implicit proposal (P0221) instead takes the view that the compiler should generate these functions for you by default.

P0221 was rejected in Oulu. The primary concerns were:

1. Many people felt that `operator<` simply does not make sense for many types, so it should not be generated by default.
2. There was no way to opt out of the operators if they were unwanted.

This proposal tries to follow the lead of P0221 while addressing the concerns that lead to it being voted down. The final result is something of a hybrid between the explicit and implicit proposals.

You may also want to read On Generating Default Comparisons, as it contains a discussion of how to support partially-ordered types.

Definition

- Support `= default` syntax to explicitly force the generation of comparison operators
- Support `= delete` syntax to define them as deleted
- All generated comparison operators have their constexpr and noexcept status deduced, just like copy constructors
- If `a` and `b` are the same type, `a == b` should be defined as equality-by-subobject if that type supports it
- If `a` and `b` are the same type, `a < b` should be defined as less-than-by-subobject if that type supports it
- If `a == b` is defined and not deleted and `a != b` is not user-defined, `a != b` should be implicitly generated as `!(a == b)`
- If `a < b` is defined and not deleted and `a > b` is not user-defined, `a > b` should be implicitly generated as `b < a`
- If `a == b` and `a < b` are defined and not deleted and `a <= b` is not user-defined, `a <= b` should be implicitly generated as `a == b or a < b`
- If `a <= b` is defined and not deleted and `a >= b` is not user-defined, `a >= b` should be implicitly generated as `b <= a`

Equality-by-subobject

A type supports equality-by-subobject if `operator==` is requested with `= default` or if the type meets all of the following requirements:

- It has no user-defined comparison operators for which both arguments are (possibly cv-qualified and possibly reference-qualified) that type
- It does not contain a mutable member
- It does not contain a virtual base class
- It does not contain a virtual member function
- It has a default copy constructor
- All base classes and non-static data members support `operator==`

If `operator==` is requested with `= default` and not all base classes and non-static data members support `operator==`, the program is ill-formed.

Equality-by-subobject is equivalent to the expression `(... and (lhs == rhs)`, where `lhs` and `rhs` are a variadic pack of references to const to all base classes and non-static data members of the type in the order in which they are defined. [note: This would presumably be worded in the same way as the copy constructor is now, and makes empty classes compare equal]

Less-than-by-subobject

A type supports less-than-by-subobject if `operator<` is requested with `= default`. If not all base classes and non-static data members support `operator<` and `operator==`, the program is ill-formed.

Less-than-by-subobject compares each base or non-static data member with `operator==`. If that comparison returns false, `operator<` returns `lhs.unequal_member < rhs.unequal_member`. If the final member is reached for a comparison, it is compared immediately with `operator<` rather than first calling `operator==`. Empty types return false for `operator<`.

Defaulting and deleting

A comparison function can be explicitly defaulted or deleted in one of the following ways

- As member function: struct A { bool operator<(A const &) const = default; };
- As a friend function: struct B { friend bool operator<(B const &, B const &) = default; };
- As a simplified declaration friend function: struct B { operator< = default; };
- As a non-member, non-friend function: struct C { }; bool operator<(C const &, C const &) = default;

For the case of the non-member, non-friend function, the `= default` or `= delete` declaration must occur in the same namespace as the class.

ODR use, hiding

The generated operators are only defined if they are ODR-used.

Using both a generated operator and a user-defined operator in the same program makes the program ill-formed. No diagnostic is required if this occurs in separate translation units.

A user-declared comparison function hides the corresponding implicitly-declared function with a similar signature (if any). Hiding means that if both appear in a lookup set, only the user-declared comparison function is considered in overload resolution. Two operator functions have a similar signature if they have the same name (for instance, `operator==`) and both arguments are of the same type (ignoring cv-qualifiers and reference-qualifiers).

Examples

Do nothing

If the user writes

```
struct A {
};
```

The following operators are defined:

```
constexpr bool operator==(A const &, A const &) noexcept {
    return true;
}
constexpr bool operator!=(A const &, A const &) noexcept {
    return false;
```

```
}
```

Error

If the user writes

```
struct B {
    A a;
};
bool operator<(B const &, B const &) = default;
```

There is a compile-time error for final line: There is no visible `operator<` for A

Generate all

If the user writes

```
struct C {
};
bool operator<(C const &, C const &) = default;
```

The following operators are defined:

```
constexpr bool operator==(C const &, C const &) noexcept {
    return true;
}
constexpr bool operator!=(C const &, C const &) noexcept {
    return false;
}
constexpr bool operator<(C const &, C const &) noexcept {
    return false;
}
constexpr bool operator>(C const &, C const &) noexcept {
    return false;
}
constexpr bool operator<=(C const &, C const &) noexcept {
    return true;
}
constexpr bool operator>=(C const &, C const &) noexcept {
    return true;
}
```

Delete

If the user writes

```
struct D1 : C {
    friend bool operator<(D1 const &, D1 const &) = delete;
};
```

The following operators are defined, if C is defined as in "Generate all":

```
constexpr bool operator==(D1 const & lhs, D1 const & rhs) noexcept {
    return static_cast<C const &>(lhs) == static_cast<C const &>(rhs);
    // equivalent to `return true;`
}
constexpr bool operator!=(D1 const & lhs, D1 const & rhs) noexcept {
    return !(lhs == rhs);
    // equivalent to `return false;`
}
bool operator<(D1 const &, D1 const &) = delete;
```

Inheritance

If the user writes

```
struct D2 : C {
};
```

The following operators are defined, if C is defined as in "Generate all":

```
constexpr bool operator==(D2 const & lhs, D2 const & rhs) noexcept {
    return static_cast<C const &>(lhs) == static_cast<C const &>(rhs);
    // equivalent to `return true;`
}
```

```
constexpr bool operator!=(D2 const & lhs, D2 const & rhs) noexcept {
    return !(lhs == rhs);
    // equivalent to `return false;`
}
```

Default and delete

If the user writes

```
struct E {
    int a;
    int b;
    std::string c;
    bool operator<(E const &) const = default;
    bool operator<=(E const &) const = delete;
};
```

The following operators are defined:

```
inline bool operator==(E const & lhs, E const & rhs) {
    return lhs.a == rhs.a and lhs.b == rhs.b and lhs.c == rhs.c;
}
inline bool operator!=(E const & lhs, E const & rhs) {
    return !(lhs == rhs);
}
bool E::operator<(E const & other) const {
    if (this->a == other.a) {
        return false;
    }
    if (this->a < other.a) {
        return true;
    }
    if (this->b == other.b) {
        return false;
    }
    if (this->b < other.b) {
        return true;
    }
    return this->c < other.c;
}
inline bool operator>(E const & lhs, E const & rhs) {
    return rhs < lhs;
}
bool operator<=(E const &, E const &) = delete;
```

Mixed-type

If the user writes

```
struct F {
    int a;
};
bool operator==(E const &, F const &) noexcept;
bool operator==(F const &, E const &) noexcept;
bool operator==(E const volatile &, F const volatile &) noexcept;
bool operator==(F const volatile &, E const volatile &) noexcept;
```

The following operators are defined, if E is defined as in "Default and delete":

```
constexpr bool operator==(F const & lhs, F const & rhs) noexcept {
    return lhs.a == rhs.a;
}
constexpr bool operator!=(F const & lhs, F const & rhs) noexcept {
    return !(lhs == rhs);
}
bool operator==(E const &, F const &) noexcept; // User-defined
bool operator==(F const &, E const &) noexcept; // User-defined
bool operator==(E const volatile &, F const volatile &) noexcept;   // User-defined
bool operator==(F const volatile &, E const volatile &) noexcept;   // User-defined
inline bool operator!=(E const & lhs, F const & rhs) noexcept {
    return !(lhs == rhs);
}
inline bool operator!=(F const & lhs, E const & rhs) noexcept {
    return !(lhs == rhs);
}
// The operators that forward to other functions perfectly forward all arguments
inline bool operator!=(E const volatile & lhs, F const volatile & rhs) noexcept {
```

```
    return !(lhs == rhs);
}
inline bool operator!=(F const volatile & lhs, E const volatile & rhs) noexcept {
    return !(lhs == rhs);
}
```

Weird return types and parameters

If the user writes

```
struct G {
};
int operator==(G const &, G &&);
double const & operator<(G, G &&) noexcept;
```

Functions equivalent to the following operators are defined:

```
int operator==(G const &, G &); // User-defined
inline bool operator!=(G const & lhs, G & rhs) {
    return !(lhs == rhs);
}
double const & operator<(G, G &&) noexcept; // User-defined
double const & operator>(G && lhs, G const & rhs) noexcept {
    return rhs < std::move(lhs);
}
double const & operator>(G && lhs, G && rhs) noexcept {
    return std::move(rhs) < std::move(lhs);
}
```

The return type of `operator!=` is bool because `operator!` applied to a value of type `int` returns type `bool`. In general, the return type of generated functions should be equivalent to `decltype(auto)`. `operator>` overloads are only defined for arguments types which are (possibly cv-qualified) G and for which the body is valid. Generated operators always use reference type arguments. `operator<=` and `operator>=` is not defined because there is no type which can be passed to both G & and G &&.

Open questions

Qualifiers

Do we allow virtual, volatile qualifiers, no const qualification, rvalue-references, or pass by value on defaulted functions? Do we allow multiple such declarations and overloading (for instance, a volatile and non-volatile `operator==`)?

Hiding

How should 'hiding' work for this proposal? Hiding is more important for implicitly-generated functions than explicitly-generated functions. An example will describe the issue best:

```
struct Base {
};
bool operator==(Base const &, Base const &);

struct Derived : Base {
    int member;
};

Derived x;
Derived y;
x == y;
```

The current behavior of `x == y` is to perform a derived-to-base conversion on both `x` and `y`, and then pass the result of that to the user-defined `bool operator==(Base const &, Base const &)`. Without a provision for hiding, this code will change meaning under this proposal to be defined as:

```
bool operator==(Derived const & lhs, Derived const & rhs) {
    return static_cast<Base const &>(lhs) == static_cast<Base const &>(rhs) and lhs.member == rhs.member;
}
```

I believe that implicitly-generated operators should be hidden by user-defined operators, as outlined in P0221. This means that if the expression `a op b` would have been valid without the implicitly-generated function, it should not change behavior and does not count as an ODR-use of the function.

The main open question here is should all the generated operators be hidden by user-defined operators, or should only implicitly-generated operators be hidden? For instance, what should be the behavior if the user in the above example had

written this instead:

```
struct Base {
};
bool operator==(Base const &, Base const &);

struct Derived : Base {
    int member;
};
bool operator==(Derived const &, Derived const &) = default;

Derived x;
Derived y;
x == y;
```

It seems reasonable that because this is the user explicitly expressing their intent that they want the generated `operator==` they should get it, and therefore the comparison should compare both `Base` and `member`. However, it also seems reasonable that a user specifying `= default` shouldn't generate a different type of thing than would be generated by default. I suspect that the least surprising option for users is that anything explicitly defaulted is not hidden and is treated the same as any other function when it comes to overload resolution.